

## tecotalk-v3

# 테코톡 대본 - 하이버네이트는 어떻게 JPQL을 SQL로 번역할까

부제: SQM에서 SQL AST까지, 하이버네이트의 쿼리 컴파일러

## 발표 설계 요약

| 항목                | 내용                                                                                                    |
|-------------------|-------------------------------------------------------------------------------------------------------|
| 핵심 메시지            | 하이버네이트는 쿼리의 의미를 담은 모델인 SQM을 먼저 만든다. 이후 그 의미를 테이블과 컬럼, 조인으로 표현할 수 있는 구조로 바꾼다.                          |
| 핵심 장면             | 같은 엔티티를 조회하더라도 상속 매핑 전략에 따라 전혀 다른 형태의 SQL이 만들어진다. lowering은 객체의 개념을 관계형 데이터베이스의 구조로 구현하는 과정이다.        |
| 도입 질문             | 자바 코드는 그대로인데, MySQL과 PostgreSQL에서 서로 다른 SQL이 실행되는 이유는 무엇일까?                                           |
| 본문에서 깊게 다루지 않을 내용 | ALL(*), SSA와 CFG, DomainResult의 세부 구조, Spring Data의 파싱 과정, 쿼리 플랜 캐시, parameter sniffing, N+1          |
| 예상 청중             | 컴파일러를 처음 접하거나 배운 내용이 가물가물한 백엔드 개발자. 방탈출 미션을 경험한 청중.                                                   |
| 기준 버전             | Spring Boot 4.1.x (Hibernate ORM 7.4.x, Spring Data JPA 4.1.x). 소스 확인과 문서 링크, 실습 로그 모두 이 버전을 기준으로 한다. |

## 시간 배분

| 순서 | 파트                          | 시간   | 누적   |
|----|-----------------------------|------|------|
| 1  | 도입: 내가 작성한 JPQL과 DB가 받은 SQL | 1:00 | 1:00 |
| 2  | 세 가지 입력이 SQM으로 모이기까지        | 1:30 | 2:30 |
| 3  | SQM은 쿼리의 의미를 담는다            | 2:00 | 4:30 |

| 순서 | 파트                          | 시간   | 누적    |
|----|-----------------------------|------|-------|
| 4  | lowering: 객체의 개념을 테이블과 조인으로 | 2:00 | 6:30  |
| 5  | 같은 조회가 세 가지 SQL로 바뀌는 이유     | 1:30 | 8:00  |
| 6  | SQL과 함께 만들어지는 결과 처리 계획      | 0:50 | 8:50  |
| 7  | 이 관점에서 N+1 살펴보기             | 0:40 | 9:30  |
| 8  | 마무리                         | 0:30 | 10:00 |

SQM을 설명하는 3파트와 lowering을 설명하는 4파트에 각각 2분을 배정한다. 이어지는 5파트와 6파트에서는 구체적인 예제를 통해 이 구조를 확인한다. 총 6분 20초를 핵심 설명에 사용하고, 나머지는 도입과 응용, 마무리에 배분한다.

표의 합계 10:00은 상한이다. 리허설에서는 9분 30초 안에 마치는 것을 목표로 하고, 각 파트를 표보다 조금씩 짧게 낭독해 30초의 여유를 만든다.

## 1. 도입: 내가 작성한 JPQL과 DB가 받은 SQL (1:00)

[슬라이드: 왼쪽에는 JPQL 한 줄, 오른쪽에는 두 DB에서 실행한 SQL을 배치한다. 달라지는 부분만 색으로 강조한다.]

여러분이 이런 JPQL을 작성했다고 해보겠습니다.

```
SELECT r FROM Reservation r WHERE r.theme.name = :name
```

여기에 페이징을 적용해서 10건만 가져오면, DB에는 이런 SQL이 전달됩니다.

```
select r1_0.id, r1_0.date, r1_0.theme_id, ...
from reservation r1_0
join theme t1_0 on t1_0.id=r1_0.theme_id
where t1_0.name=?
limit ?
```

저는 join 을 쓴 적이 없습니다. r1\_0 이라는 별칭을 붙인 적도 없고요.

이번에는 자바 코드를 그대로 두고, DB만 MySQL에서 PostgreSQL로 바꿔보겠습니다. 그러면 마지막 부분이 이렇게 달라집니다.

```
fetch first ? rows only
```

같은 JPQL인데, 실행되는 SQL은 다릅니다. 하이버네이트가 제가 작성한 쿼리의 의미를 해석하고, 각 DB에 맞는 문법으로 SQL을 만들어주기 때문입니다.

그렇다면 이 과정은 하이버네이트 안에서 어떻게 이루어질까요?

오늘은 그 안을 들여다보겠습니다. 하이버네이트에는 입력을 해석하는 프론트엔드, 해석한 내용을 담는 중간 표현, SQL을 만들어내는 백엔드가 있습니다. 이 구조를 따라가 보면, 하이버네이트 안에 있는 **쿼리 컴파일러**를 만날 수 있습니다.

**진행 팁:** 도입에서는 N+1을 언급하지 않는다. 오늘 발표가 쿼리의 처리 구조를 살펴보는 시간이라는 점을 먼저 전달한다.

**사전 확인:** 도입 예제는 `setMaxResults(10)` 만 적용한 조회로 고정한다(실습 2와 같은 조건). 하이버네이트 7.4는 시작 위치가 지정됐을 때만 `offset`을 출력한다. `Pageable` 이나 `setFirstResult` 를 함께 쓰면 MySQL은 `limit ?,?`, PostgreSQL은 `offset ? rows fetch first ? rows only` 처럼 양쪽이 모두 바뀔 수 있어, "마지막 한 줄만 다르다"는 장면이 흐려진다. 두 SQL은 실제 MySQL과 PostgreSQL 환경에서 얻은 로그로 교체한다.

## 2. 세 가지 입력이 SQM으로 모이기까지 (1:30)

### 2-1. 쿼리를 만드는 세 가지 방법 (50초)

[슬라이드: 메서드 이름, HQL 문자열, Criteria API 코드를 나란히 보여준다.]

우리는 쿼리를 여러 방식으로 만듭니다. 여기서는 세 가지를 보겠습니다.

```
// A. 메서드 이름
List<Reservation> findByThemeName(String name);

// B. HQL 문자열
em.createQuery("SELECT r FROM Reservation r WHERE r.theme.name = :name",
Reservation.class);

// C. Criteria API
cq.select(r).where(cb.equal(r.get("theme").get("name"), name));
```

먼저 A의 `findByThemeName` 을 보겠습니다. 쿼리를 직접 쓰는 대신, 메서드 이름에 조회 조건을 담았습니다. 스프링 데이터는 이 이름에서 조회 대상과 조건을 읽어내 쿼리를 만듭니다.

이처럼 이름을 일정한 규칙에 따라 나누고, 그 구조를 해석하는 것도 파싱입니다. 우리는 메서드 이름으로 쿼리를 만들 때마다 파서를 사용하고 있었던 셈이죠.

C는 접근 방식이 다릅니다. Criteria API에서는 메서드 호출로 쿼리의 구조를 직접 조립합니다. HQL 문자열을 렉서와 파서에 넘기는 단계가 필요하지 않습니다.

입력 방식은 이렇게 다르지만, 하이버네이트 안에서는 공통된 표현으로 모입니다.

## 2-2. 공통된 표현, SQM (40초)

[슬라이드: 메서드 이름이 JPQL 문자열로 바뀌어 HQL 입구로 들어가는 경로를 위에 놓고, HQL과 Criteria가 SQM으로 모이는 구조를 그린다.]

그 표현이 바로 **SQM, Semantic Query Model**입니다.

B의 HQL 문자열은 ANTLR 4로 파싱합니다. 저장소에는 `HqlParser.g4` 라는 문법 파일이 있고, 파싱 결과를 `SemanticQueryBuilder` 가 읽으면서 SQM을 만듭니다.

C의 경우는 더 직접적입니다. 하이버네이트에서 Criteria API로 조립하는 트리 자체가 SQM입니다. `CriteriaBuilder` 의 구현체 이름도 `SqmCriteriaNodeBuilder` 죠.

그렇다면 A는 어떻게 연결될까요? `PartTreeJpaQuery` 가 메서드 이름을 해석하고, `JpaQueryCreator` 가 그 결과로 JPQL 문자열을 만듭니다. 그 문자열이 B와 같은 입구로 들어가는 구조입니다.

그러니까 메서드 이름 쿼리는 **두 번 파싱됩니다**. 한 번은 스프링 데이터가 메서드 이름을, 또 한 번은 하이버네이트가 그렇게 만들어진 JPQL을 파싱하는 거죠.

[실습 1 결과 삽입: 세 경로에서 생성된 SQM 덤프를 비교하고, 공통된 구조를 색으로 표시한다.]

**진행 팁:** 메서드 이름이 JPQL 문자열을 거쳐 HQL 경로로 합류하는 흐름이 드러나도록 그린다. 세 입력을 나란한 세 갈래로 그리지 않는다. 서로 다른 입력이 공통된 중간 표현으로 모인다는 점에 초점을 맞춘다.

**진행 팁:** Criteria를 두고 왜 굳이 문자열을 만드느지는 본문에서 설명하지 않는다. 질문이 나오면 B-5로 답한다.

**진행 팁:** `SqmCriteriaNodeBuilder` 와 `SqmFrom` 의 인터페이스 계층을 보조 자료로 준비한다. Criteria API로 SQM을 직접 조립한다는 설명을 실제 코드와 연결할 수 있다.

**사전 확인:** Spring Data JPA 4.1에서 `JpaQueryCreator` 가 `JpqlQueryBuilder` 로 JPQL 문자열을 만들고, `PartTreeJpaQuery` 가 그 문자열을 `EntityManager.createQuery` 에 넘기는 경로를 스택 트레이스로 확인한다(실습 1). 슬라이드는 확인한 경로를 기준으로 그린다. Spring Data JPA 3.x는 같은 자리에서 Criteria 쿼리를 만들었으므로, 버전을 바꾸면 합류 지점도 바뀐다.

### 3. SQM은 쿼리의 의미를 담는다 (2:00)

#### 3-1. 이름에 Semantic이 들어가는 이유 (40초)

[슬라이드: `r.theme.name` 을 구문 트리와 SQM으로 각각 표현해 비교한다.]

Semantic Query Model이라는 이름에서 눈여겨볼 단어는 **Semantic**, 즉 의미입니다.

`r.theme.name` 을 예로 들어보겠습니다.

파싱 단계에서는 이것을 점으로 연결된 세 개의 식별자로 읽습니다. 이 단계의 역할은 표현이 문법에 맞는지 확인하고, 구문 구조를 만드는 것입니다.

SQM에는 여기에 의미 분석 결과가 더해집니다. `r` 은 `Reservation` 엔티티를 가리킵니다. `theme` 는 `Theme` 엔티티와 연결된 연관 속성이고, `name` 은 그 엔티티의 `String` 타입 속성입니다.

즉, SQM에는 각 이름이 무엇을 가리키고, 어떤 타입을 갖는지까지 담겨 있습니다.

#### 3-2. 오타를 내보면 차이가 보인다 (40초)

[슬라이드: 오타가 있는 JPQL, 발생한 예외, SQL이 실행되지 않은 DB 로그를 함께 보여준다.]

이 차이는 오타를 내보면 확인할 수 있습니다.

```
em.createQuery("SELECT r FROM Reservation r WHERE r.theme.nmae = :name");
```

`name` 을 `nmae` 로 잘못 적었습니다. 예외가 발생하는데, DB 로그에는 실행된 SQL이 없습니다. DB에 쿼리를 보내기 전에 하이버네이트가 오류를 발견한 겁니다.

이 표현은 문법적으로는 문제가 없습니다. 점 뒤에 식별자가 왔으니까요. 그런데도 오류라고 판단하려면, **Theme 에 nmae 라는 속성이 없다는 사실**을 알아야 합니다. 이것이 의미 분석입니다.

타입 정보도 함께 다룹니다. `SqmExpression<T>` 같은 표현식 노드를 통해, 각 표현식이 어떤 타입의 값을 나타내는지 관리합니다.

[실습 3 결과 삽입: 예외 스택 트레이스와 SQL이 실행되지 않은 로그를 한 화면에 배치한다.]

**진행 팁:** 입력 방식에 따라 오류가 드러나는 시점이 다르다는 점을 짧게 덧붙일 수 있다. 메서드 이름을 해석할 때, Criteria를 조립할 때, HQL을 해석할 때를 비교하되, 실제 프로젝트에서 확인한 범위만 설명한다.

### 3-3. 중간 표현을 두는 이유 (40초)

[슬라이드: 입력과 DB를 각각 직접 연결한 그림과, 공통된 중간 표현을 거치는 그림을 비교한다.]

입력 방식이 여러 가지인 것처럼, 지원해야 할 DB도 여러 가지입니다. MySQL, PostgreSQL, Oracle, H2가 있죠.

입력마다 각 DB로 직접 번역한다면 모든 조합을 처리해야 합니다. 반면 공통된 중간 표현을 두면, 입력을 해석하는 부분과 DB에 맞게 출력하는 부분을 나눌 수 있습니다.

흔히 M 곱하기 N의 문제를 M 더하기 N의 문제로 바꾼다고 설명합니다. 새로운 입력 방식이나 DB를 지원할 때, 기존의 모든 조합을 다시 구현할 필요가 줄어드는 겁니다.

Clang과 Rust 컴파일러가 서로 다른 언어를 처리하면서 LLVM IR을 활용하는 구조도 비슷합니다.

이런 관점에서 SQM은 **고수준 중간 표현, high-level IR에 해당하는 역할**을 합니다. 엔티티와 속성처럼, 우리가 쿼리를 작성할 때 사용한 개념을 유지하고 있기 때문입니다.

[슬라이드용 대응표: 두 흐름의 역할을 비교하기 위한 개략도]

```
[컴파일러] 소스 코드 → 렉서 → 파서 → 구문 트리 → 의미 분석 → IR → 백엔드 → 기계어
[Hibernate] HQL      → 렉서 → 파서 → 구문 트리 → 의미 분석 → SQM → 백엔드 → SQL
```

**진행 팁:** 여기서는 SQM이 맡는 역할을 컴파일러의 중간 표현에 빗대어 설명한다. 프론트엔드와 백엔드를 분리한다는 특징만으로 IR의 정의를 설명하려 하지 않는다.

## 4. lowering: 객체의 개념을 테이블과 조인으로 (2:00)

### 4-1. 중간 표현이 두 단계인 이유 (40초)

[슬라이드: 위에는 SQM, 아래에는 SQL AST를 놓고, 둘 사이를 lowering 화살표로 연결한다.]

지금까지 SQM을 살펴봤는데요. SQL을 만들기까지는 중간 표현이 하나 더 있습니다. **SQL AST**입니다.

둘의 차이는 무엇을 표현하느냐에 있습니다.

| 구분         | SQM                                  | SQL AST                                              |
|------------|--------------------------------------|------------------------------------------------------|
| 관점         | 객체 모델                                | 관계형 데이터 모델                                           |
| 주요 요소      | 엔티티, 속성, 연관관계                        | 테이블, 컬럼, JOIN                                        |
| 표현하는 내용의 예 | <code>r.theme.name</code>            | <code>theme</code> 테이블과의 조인, <code>name</code> 컬럼 참조 |
| 패키지        | <code>org.hibernate.query.sqm</code> | <code>org.hibernate.sql.ast</code>                   |

SQM은 엔티티와 연관관계로 쿼리를 표현합니다. SQL AST는 이를 테이블과 컬럼, 조인으로 풀어냅니다.

이처럼 추상적인 표현을 더 구체적인 표현으로 바꾸는 과정을 컴파일러에서는 **lowering**이라고 부릅니다. 하이버네이트에서는 `StandardSqmTranslator`를 통해 이 과정을 수행합니다.

## 4-2. 연관관계가 조인으로 표현된다 (40초)

[슬라이드: `r.theme.name` 이 조인과 컬럼 참조로 바뀌는 과정을 보여준다.]

다시 `r.theme.name`을 보겠습니다. 자바의 객체를 따라가듯이 쓴 짧은 경로입니다.

하지만 DB에서 같은 값을 찾으려면 구체적인 방법이 필요합니다. `reservation` 테이블과 `theme` 테이블을 어떤 키로 연결하고, 어느 컬럼을 읽을지 정해야 하죠.

SQL AST에는 이 내용이 조인 조건과 컬럼 참조로 표현됩니다. `r.theme.store.name`처럼 연관관계를 더 따라가면, 그 경로에 필요한 조인도 추가됩니다.

우리가 짧게 표현한 연관관계가, DB에서 실행할 수 있는 구조로 풀리는 겁니다. **추상적인 개념을 구체적인 구조로 바꾼다.** 이것이 lowering의 핵심입니다.

**사전 확인:** 실제 매핑과 쿼리에서 어떤 조인이 생성되는지 로그로 확인한다. 이 예제의 HQL 경로 표현식에 따른 암묵적 연관 조인은 내부 조인으로 설명한다. 명시적 조인이나 페치 조인의 동작과 혼동하지 않는다. 7.4 가이드에도 암묵적 조인은 항상 내부 조인으로 다룬다고 적혀 있다. [Hibernate 7.4 HQL 가이드의 암묵적 조인 설명](#)

## 4-3. DB에 맞는 SQL이 만들어지는 단계 (40초)

[슬라이드: 도입의 JPQL과 SQL 비교 화면에 SQL AST와 변환 단계를 추가한다.]

이제 처음의 질문으로 돌아가 보겠습니다. 같은 자바 코드인데, 왜 DB를 바꾸면 SQL이 달라질까요?

SQL AST는 아직 트리 형태입니다. 이것을 SQL 문자열로 출력하는 단계가 따로 있고, 그 역할을 하는 인터페이스가 `SqlAstTranslator`입니다.

구현체 이름을 보면 역할이 더 분명해집니다.

```
MySQLSqlAstTranslator
PostgreSQLSqlAstTranslator
OracleSqlAstTranslator
SQLServerSqlAstTranslator
...
```

DB별로 SQL을 생성하는 구현체가 있는 겁니다. 어떤 구현체를 사용할지는 `Dialect` 와 연결됩니다.

컴파일러가 x86과 ARM에 맞춰 서로 다른 명령어를 만드는 것처럼, 하이버네이트도 DB에 맞는 SQL 문법을 선택합니다.

도입에서 봤던 페이징 문법의 차이도 이 과정에서 설명할 수 있습니다. **조회하려는 내용은 같아도, 그것을 표현하는 SQL 문법은 DB에 따라 달라지는 거죠.**

**[실습 2 결과 삽입: MySQL과 PostgreSQL의 실제 SQL을 좌우로 비교한다.]**

**진행 팁:** 도입에서 던진 질문에 답하는 부분이다. `SqlAstTranslator` 구현체 이름을 보여준 뒤 잠시 쉬어간다. 이 예제에서 확인한 차이를 설명하고, 모든 DB 차이가 마지막 출력 단계에서만 발생한다고 일반화하지 않는다.

## 5. 같은 조회가 세 가지 SQL로 바뀌는 이유 (1:30)

**[슬라이드: `SlotClaim` 조회에서 세 가지 SQL 형태로 갈라지는 그림을 보여준다.]**

이번에는 한 단계 더 들어가 보겠습니다. 객체 모델의 개념을 SQL로 표현하려면 어떤 일이 필요할까요?

방탈출 미션에서 예약과 대기를 설계할 때, 공통된 정보를 어떻게 관리할지 고민해보셨을 겁니다. 여기서는 둘을 `SlotClaim` 이라는 공통 부모로 묶어보겠습니다.

```

@Entity
@Inheritance(strategy = ???)
abstract class SlotClaim {
    @ManyToOne Member member;
    @ManyToOne Theme theme;
    LocalDate date;
}

@Entity class Reservation extends SlotClaim { }
@Entity class Waiting extends SlotClaim { }

```

내 예약과 대기를 한 번에 조회하는 JPQL은 이렇게 쓸 수 있습니다.

```
SELECT c FROM SlotClaim c WHERE c.member.id = :memberId
```

SQM에서는 `SlotClaim` 을 조회 대상으로 표현합니다. 하지만 실제 데이터가 어느 테이블에 저장되는지는 상속 매핑 전략에 따라 다릅니다.

| 전략              | 생성되는 SQL의 주요 구조              |
|-----------------|------------------------------|
| SINGLE_TABLE    | 하나의 테이블을 조회하고 판별 컬럼으로 타입을 구분 |
| JOINED          | 부모 테이블과 자식 테이블을 JOIN         |
| TABLE_PER_CLASS | 구체 클래스의 테이블을 UNION ALL로 연결   |

**[실습 4 결과 삽입: 전략별 SQL의 주요 부분과 SQM의 공통된 조회 구조를 함께 보여준다.]**

JPQL에서 요청한 내용은 같습니다. 그런데 상속 전략을 바꾸면 SQL의 구조 자체가 달라집니다.

컴파일러에서도 비슷한 일이 일어납니다. C의 `switch` 하나가 상황에 따라 점프 테이블로 구현되기도 하고, 여러 조건 분기로 풀리기도 하죠.

하이버네이트도 같은 원리로 이해할 수 있습니다. 객체 모델에서 표현한 의미를, 테이블과 조인처럼 DB가 처리할 수 있는 구조로 구현하는 것. 이것이 lowering이 하는 일입니다.

**진행 팁:** 세 SQL을 모두 읽지 않는다. 단일 테이블, JOIN, UNION ALL이라는 구조의 차이만 짚는다. 상속 매핑에 따라 SQL의 형태가 달라지는 장면에 시간을 배정한다.

**슬라이드 주의:** 실습 코드에서는 이름 충돌을 피하기 위해 `SlotReservation`, `SlotWaiting` 을 사용할 수 있다. 슬라이드에서는 `Reservation`, `Waiting` 으로 단순화하되, 실제 코드와의 차이를 설명할 수 있도록 준비한다.

**사전 확인:** SQM 덤프를 비교해 어떤 조회 구조가 공통으로 유지되는지 확인한다. 조회 전체가 노드 하나로 이루어졌다는 인상을 주지 않도록, `SlotClaim` 을 가리키는 조회 대

상과 쿼리 전체의 구조를 구분해 설명한다.

## 6. SQL과 함께 만들어지는 결과 처리 계획 (0:50)

[슬라이드: 여러 결과 행이 하나의 예약 객체와 대기 컬렉션으로 구성되는 그림을 보여준다.]

하이버네이트가 준비하는 것은 SQL만이 아닙니다. 다음 페치 조인을 보겠습니다.

```
SELECT r FROM Reservation r JOIN FETCH r.waitings
```

대기가 세 개 연결된 예약 한 건을 조회하면, SQL 결과에는 세 개의 행이 나옵니다. 자바에서 필요한 결과는 예약 객체 하나와, 그 안에 대기 객체 세 개가 들어 있는 컬렉션이죠.

그래서 쿼리를 변환할 때는 두 가지를 함께 준비합니다.

- **SQL AST**: DB에서 어떤 데이터를 조회할지 표현한 구조
- **DomainResult 그래프**: 조회한 결과를 어떤 객체로 구성할지 표현한 계획

어느 컬럼을 예약의 날짜로 읽고, 어느 컬럼들을 대기 객체로 묶을지 미리 정해두는 겁니다. 쿼리를 실행한 뒤에는 이 계획에 따라 결과를 객체로 구성합니다.

컴파일러가 실행 코드와 함께 실행에 필요한 메타데이터를 준비하는 것에 비유할 수 있습니다. 무엇을 조회할지와, 조회한 결과를 어떻게 사용할지를 함께 준비하는 거죠.

[실습 5 결과 삽입: SQL AST와 DomainResult 덤프를 함께 보여준다.]

**진행 팁**: 여기서 말하는 컴파일 시점은 하이버네이트가 쿼리를 해석하고 실행을 준비하는 시점이다. 자바 소스 코드를 컴파일하는 시점과 구분한다.

## 7. 이 관점에서 N+1 살펴보기 (0:40)

[슬라이드: 전체 처리 흐름에 쿼리를 준비하는 시점과 객체에 접근하는 시점을 표시한다.]

이 구조를 알고 나면 N+1도 조금 다르게 볼 수 있습니다.

하이버네이트가 쿼리 실행 계획을 준비할 때는, 그 뒤의 자바 코드에서 어떤 연관 객체에 접근할지 모두 알 수 없습니다. 반복문 안에서 `getTheme()` 을 호출하면서 지연 로딩이 발생하면, 추가 쿼리가 실행될 수 있죠.

JOIN FETCH 나 @EntityGraph 는 함께 가져올 데이터를 미리 지정하는 방법입니다.  
@BatchSize 는 추가 조회를 여러 건씩 묶을 수 있도록 설정하는 방법이고요.

동작 방식에는 차이가 있지만, 필요한 데이터와 조회 방법을 하이버네이트에 알려준다는 관점에서 이해할 수 있습니다.

오늘은 쿼리가 변환되는 과정에 집중하겠습니다. N+1에 관한 자세한 내용은 질의응답에서 이어가겠습니다.

**진행 팁:** 40초 안에 마무리한다. 해결 방법을 하나씩 설명하기 시작하면 발표의 초점이 바뀔 수 있다. 마지막 문장으로 범위를 정리하고 마무리로 넘어간다.

## 8. 마무리 (0:30)

[슬라이드: 입력부터 SQM, SQL AST, DB별 SQL 생성까지의 흐름을 한 장에 보여준다.]

오늘 살펴본 내용을 정리하겠습니다.

하이버네이트는 먼저 **쿼리의 의미를 담은 SQM**을 만듭니다.

그다음 엔티티와 연관관계를 **테이블과 조인으로 풀어내고**, DB에 맞는 SQL을 생성합니다.

입력을 해석하는 프론트엔드부터 중간 표현, SQL을 생성하는 백엔드까지, 하이버네이트 안에는 **쿼리 컴파일러**가 있습니다.

익숙하게 사용하던 프레임워크 안에서도, 컴파일러의 기본 원리가 작동하고 있었던 겁니다. 감사합니다.

**진행 팁:** 마무리는 문장째로 익혀둔다. 시간이 부족하더라도 SQM, lowering, 쿼리 컴파일러라는 세 가지 핵심이 남도록 한다.

## 예상 질문과 답변

### A. SQM과 중간 표현

SQM을 컴파일러의 중간 표현에 비유한 이유와, 그 비유가 어디까지 유효한지 설명하기 위한 답변이다.

## A-0. "SQM을 정말 IR이라고 볼 수 있나요?"

발표에서는 SQM이 **고수준 중간 표현에 해당하는 역할을 한다**고 설명했습니다. 입력한 쿼리와 최종 SQL 사이에서 어떤 역할을 맡는지에 초점을 맞춘 표현입니다.

SQM은 쿼리의 구조와 타입 정보를 담고, 서로 다른 입력 경로가 공통으로 사용할 수 있는 표현을 제공합니다. 이후에는 이를 SQL AST로 변환할 수 있습니다.

이처럼 입력을 해석하는 단계와 SQL을 만드는 단계를 연결한다는 점에서, 컴파일러의 중간 표현에 비유할 수 있습니다. 다만 프론트엔드와 백엔드를 분리한다는 특징 하나만으로 IR의 정의를 설명하려는 것은 아닙니다.

**답변 요령:** 발표에서 사용한 비유의 목적과 범위를 먼저 설명한다. 용어의 정의를 두고 논쟁하기보다, SQM이 실제로 맡는 역할로 답변을 이어간다.

## A-1. "SQM은 어느 수준의 IR에 해당하나요?"

고수준 IR에 가깝다고 볼 수 있습니다. 엔티티와 속성, 연관관계처럼 쿼리를 작성할 때 사용한 개념과 타입 정보를 유지하기 때문입니다.

예를 들어 `SqmExpression<T>` 처럼 표현식의 타입을 다루는 구조가 있습니다. 쿼리의 구문 구조에 의미 분석 결과가 더해져 있는 셈입니다.

SQL AST는 그보다 SQL에 가까운 표현입니다. SQM이 엔티티를 중심으로 표현한다면, SQL AST는 테이블과 컬럼을 중심으로 표현하죠. 특정 컴파일러의 IR과 정확히 일대일로 대응시키기보다는, 추상화 수준이 다른 두 단계로 이해하면 좋겠습니다.

## A-2. "왜 중간 표현이 두 개나 필요한가요? SQM에서 바로 SQL을 만들면 안 되나요?"

두 단계가 해결하는 문제가 다르기 때문입니다.

SQM에서 SQL AST로 넘어갈 때는 객체 모델을 관계형 데이터 모델로 풀어냅니다. 상속을 여러 테이블로 표현하거나, 연관관계를 조인으로 바꾸는 일이 여기에 해당합니다.

SQL AST에서 SQL 문자열을 만들 때는 DB가 지원하는 문법을 반영합니다. 페이징을 어떤 문법으로 표현할지가 대표적인 예죠.

이 역할을 나누면 객체 매핑을 처리하는 로직과 DB별 SQL을 출력하는 로직을 각각 재사용하기 쉬워집니다. 하이버네이트 6에서 들어와 7.4까지 이어지는 쿼리 처리 구조를 이해할 때도 이 구분이 중요합니다.

### A-3. "SQM에도 최적화 과정이 있나요?"

하이버네이트의 쿼리 처리 과정에도 불필요한 작업을 줄이는 처리가 있습니다. 예를 들어 연관 엔티티의 식별자만 필요하다면, 매핑에 따라 조인 없이 외래 키 컬럼을 활용할 수 있습니다.

다만 이런 처리를 모두 SQM 자체의 최적화라고 보기보다는, SQM에서 SQL을 만들어가는 여러 단계에서 이루어지는 처리로 구분하는 편이 정확합니다.

조인 순서나 인덱스 선택처럼 실제 실행 방법을 결정하는 일은 주로 DB 옵티마이저가 담당합니다. 하이버네이트는 객체 모델의 조회 요청을 SQL로 표현하고, DB는 그 SQL을 어떻게 실행할지 결정하는 구조입니다.

N+1을 볼 때도 이 경계를 구분해야 합니다. 개별 SQL의 실행 계획과, 애플리케이션이 여러 SQL을 추가로 실행하는 문제는 서로 다른 층에서 발생합니다.

### A-4. "그럼 SSA나 CFG 같은 것도 있나요?"

오늘 살펴본 SQM과 SQL AST는 SSA나 CFG를 중심으로 구성된 표현은 아닙니다.

CFG는 명령의 제어 흐름을, SSA는 값의 정의와 사용 관계를 다루는 데 쓰입니다. 반복문과 분기가 있는 프로그램을 분석할 때 자주 등장하죠.

반면 SQM은 어떤 데이터를 조회할지 표현하는 모델입니다. SQL의 논리적 구조를 설명할 때는 관계 대수나 쿼리 트리가 더 가까운 비유입니다. 다루는 대상이 다르기 때문에 중간 표현의 형태도 달라지는 겁니다.

**답변 요약:** SQM의 구조에 관한 답변과 DB 내부의 실행 엔진에 관한 답변을 구분한다. SQL을 처리하는 모든 시스템에 제어 흐름이나 SSA가 없다고 범위를 넓혀 단정하지 않는다.

### A-5. "ANTLR 4는 어떤 파싱 알고리즘을 사용하나요?"

**ALL(\*), Adaptive LL-star**를 사용합니다.

LL(k)는 다음 토큰을 정해진 개수인 k개까지 보고 문법의 어느 대안을 선택할지 판단합니다. ALL(\*)는 필요한 만큼 입력을 살펴보면서 판단할 수 있도록 이 방식을 확장합니다.

선택이 필요한 지점에서 가능한 대안들을 입력과 비교하고, 그 과정에서 얻은 정보를 DFA로 저장해 재사용합니다.

이론적인 최악의 시간 복잡도는 입력 길이의 4제곱이지만, 논문에서는 실용적인 문법에 대해 대체로 선형적인 성능을 보였다고 설명합니다. 표현력과 실제 성능을 함께 고려한 방식으로 이해하면 됩니다. [ANTLR의 ALL\(\\*\) 논문](#)

**답변 요령:** 알고리즘의 특징과 하이버네이트가 ANTLR 4를 채택한 이유는 구분한다. 채택 이유를 직접 확인하지 않았다면 속도 때문이라고 추정해서 설명하지 않는다.

## A-6. "DomainResult 그래프도 IR인가요?"

이 발표에서는 쿼리를 처리하면서 함께 만들어지는 결과 매핑 계획으로 설명했습니다.

SQL AST가 어떤 데이터를 조회할지 표현한다면, DomainResult 그래프는 조회한 데이터를 어떤 결과로 구성할지 표현합니다.

컴파일러에 비유하면 실행 코드와 함께 준비하는 메타데이터에 가깝습니다. IR이라는 이름을 붙일 수 있는지보다, SQL 실행 결과를 객체로 구성하는 데 필요한 정보를 담는다는 역할이 중요합니다.

## B. 입력과 파싱

### B-1. "Criteria가 SQM이라면, Criteria에는 프론트엔드가 없는 건가요?"

문자열을 파싱하는 단계 없이, API로 중간 표현을 조립하는 방식이라고 보면 됩니다.

하이버네이트의 CriteriaBuilder 구현체가 SqmCriteriaNodeBuilder 이고, Criteria API로 만드는 노드가 SQM을 구성합니다. 개발자가 API를 통해 쿼리의 구조를 직접 전달하는 겁니다. [Hibernate 7.4의 Criteria 설명](#)

컴파일러에서도 API를 통해 IR을 직접 만드는 경우가 있습니다. LLVM의 IRBuilder 에 비유하면 이해하기 쉽습니다. 소스 문자열을 먼저 만들지 않고도 구조화된 표현을 조립할 수 있다는 점이 비슷합니다.

### B-2. "QueryDSL은 컴파일할 때 오류를 잡아주지 않나요?"

네. Q 클래스를 통해 속성 이름과 타입을 자바 코드로 표현하기 때문에, 자바 컴파일러가 확인할 수 있는 오류가 늘어납니다.

예를 들어 존재하지 않는 필드를 사용하면 Q 클래스에 해당 필드가 없으므로 컴파일 오류가 납니다. 문자열로 쿼리를 작성할 때보다 이름과 타입을 미리 확인하기 쉬워지는 거죠.

다만 QueryDSL JPA로 작성한 쿼리도 실행할 때는 JPA 구현체의 쿼리 처리 과정을 거칩니다. 하이버네이트를 사용한다면 SQL로 변환하는 과정은 여전히 애플리케이션 실행 중에 이루어 집니다.

컴파일러의 관점에서 보면, QueryDSL은 쿼리를 자바의 타입 시스템이 검사할 수 있는 형태로 표현하도록 도와주는 도구라고 할 수 있습니다.

### B-3. "Spring Data의 메서드 이름 쿼리는 어디에서 합류하나요?"

발표 환경인 Spring Data JPA 4.1에서는 `PartTreeJpaQuery` 가 `PartTree` 를 이용해 메서드 이름을 해석하고, `JpaQueryCreator` 가 `JpqlQueryBuilder` 로 JPQL 문자열을 만듭니다.

이 문자열을 `EntityManager.createQuery` 에 넘기므로, 그다음은 HQL 문자열과 같은 경로입니다. 그래서 슬라이드에서도 메서드 이름이 JPQL 문자열을 거쳐 HQL 입구로 들어가도록 그렸습니다.

Spring Data JPA 3.x까지는 같은 자리에서 Criteria 쿼리를 만들었습니다. 버전에 따라 합류 지점이 달라지는 셈입니다. 3.x에서는 Criteria 쪽으로, 4.x에서는 HQL 쪽으로 합류합니다.

**사전 확인:** 이 답변은 발표 환경에서 해당 경로를 스택 트레이스로 확인한 뒤 사용한다. `@Query` 나 `Specification`처럼 Spring Data가 제공하는 다른 쿼리 작성 방식까지 같은 경로로 일반화하지 않는다.

### B-4. "네이티브 쿼리는 어떻게 처리하나요?"

네이티브 쿼리는 개발자가 SQL을 직접 작성하므로, 오늘 살펴본 HQL에서 SQM을 거쳐 SQL을 만드는 경로를 우회합니다.

그만큼 SQL의 테이블명과 컬럼명, DB별 문법을 직접 관리해야 합니다. 엔티티 매핑이나 상속 전략을 바꿨을 때도 작성한 SQL이 여전히 맞는지 확인해야 하죠.

컴파일러의 인라인 어셈블리에 비유할 수 있습니다. 더 직접적으로 제어할 수 있는 대신, 상위 추상화가 처리해주던 부분을 개발자가 맡게 됩니다.

### B-5. "Criteria로 조립하면 파싱을 건너뛸 수 있는데, Spring Data는 왜 굳이 문자열을 만드나요?"

Spring Data JPA는 4.0에서 메서드 이름 쿼리를 Criteria 대신 JPQL 문자열로 만들도록 바꿨습니다. 해당 이슈에는 하이버네이트가 문자열 쿼리를 훨씬 효율적으로 실행한다는 벤치마크가 근거로 적혀 있습니다. 초당 처리량이 약 3배 차이였습니다. [spring-data-jpa 이슈 #3588](#)

하이버네이트 7.4의 설정을 보면 이유 하나를 짐작할 수 있습니다. 쿼리 플랜 캐시는 기본적으로 켜져 있지만, Criteria 쿼리의 해석 결과를 캐시하는

`hibernate.criteria.plan_cache_enabled` 는 기본값이 꺼져 있습니다(7.0에서 추가된 설정). 같은 JPQL 문자열은 두 번째 실행부터 해석 결과를 재사용할 수 있지만, Criteria 쿼리는 기본 설정에서 실행할 때마다 다시 해석됩니다.

파싱을 한 번 건너뛰는 이득보다, 한 번 해석한 결과를 계속 재사용하는 이득이 클 수 있다는 뜻입니다.

[Q&A 보조 슬라이드: 실습 8에서 측정한 HQL과 Criteria의 플랜 캐시 적중 결과]

**답변 요령:** 벤치마크 수치는 이슈에 적힌 값으로만 인용한다. 3배 차이의 원인이 플랜 캐시 하나뿐이라고 단정하지 않고, 실습 8에서 확인한 범위만 말한다.

## B-6. "Spring Data도 JPQL을 파싱하나요?"

네. Spring Data JPA는 자체 JPQL 문법(Jpql.g4)을 가지고 있습니다. @Query 로 받은 문자열에 정렬을 덧붙이거나, 페이징에 필요한 count 쿼리를 만들어내기 위해서입니다.

다만 목적이 다릅니다. 쿼리를 SQM으로 만드는 파서는 하이버네이트 쪽에 있고, Spring Data의 파서는 문자열을 고쳐 쓰기 위한 파서입니다.

[실습 1의 "여유가 되면" 항목에서 확인한 내용으로 보완한다.]

**답변 요령:** 메서드 이름 쿼리(B-3)와 @Query 문자열 처리를 섞지 않는다. Spring Data 파서가 어느 경로에서 쓰이는지는 확인한 범위만 말한다.

## C. 실무에서 연결되는 질문

### C-1. "본문에서 짧게 다룬 N+1을 조금 더 설명해주실 수 있나요?"

지연 로딩과 즉시 로딩을 나눠서 보면 이해하기 쉽습니다.

**지연 로딩에서의 N+1**은 처음 조회할 때 가져오지 않은 연관 데이터에 접근하면서 발생할 수 있습니다. 목록을 조회한 뒤 반복문에서 각 항목의 연관 객체를 읽으면, 추가 쿼리가 이어지는 겁니다.

**즉시 로딩에서도 N+1이 발생할 수 있습니다.** EAGER는 연관 데이터를 언제 준비해야 하는지를 정하는 설정입니다. 반드시 하나의 조인 쿼리로 가져온다는 뜻은 아닙니다.

예를 들어 JPQL에 페치 조인이 없으면, 하이버네이트가 결과를 반환하기 전에 추가 조회로 EAGER 연관관계를 채울 수 있습니다. 따라서 EAGER 설정만으로 N+1이 해결되지는 않습니다. [Hibernate 7.4의 페치 전략 설명](#)

findById 와 JPQL 조회에서 SQL이 달라질 수 있는 이유도, 조회 경로에 따라 연관 데이터를 가져오는 방식이 달라질 수 있기 때문입니다. 실제 동작은 매핑과 실행 환경에서 확인해야 합니다.

[Q&A 보조 슬라이드: 실습 7에서 측정한 다섯 조건의 SQL과 쿼리 수를 비교한다.]

**답변 요약:** 실제로 측정된 조건과 결과를 먼저 제시한다. LAZY 또는 EAGER라는 설정 하나만으로 모든 SQL의 형태를 설명하려 하지 않는다.

## C-2. "그럼 항상 JOIN FETCH를 사용하면 되지 않나요?"

페치 조인도 조회 목적에 맞게 사용해야 합니다. 대표적으로 세 가지를 고려해야 합니다.

1. **결과 행이 크게 늘어날 수 있습니다.** 여러 컬렉션을 한꺼번에 페치 조인하면, 각 컬렉션의 원소가 조합되면서 결과 행 수가 급격히 증가할 수 있습니다.
2. **컬렉션 페치 조인과 페이징을 함께 쓸 때는 버전과 DB를 확인해야 합니다.** SQL의 행 수 제한과 우리가 원하는 엔티티 수 제한은 다릅니다. 하이버네이트 7.4 이전에는 전체 결과를 읽은 뒤 메모리에서 페이징했습니다. 7.4부터는 서브쿼리 안에서 limit과 offset을 지원하는 DB라면 이 문제가 해결됐고, 지원하지 않는 DB에서는 여전히 메모리에서 페이징합니다.
3. **사용하지 않을 데이터까지 가져올 수 있습니다.** 전송량과 객체 생성 비용이 늘어납니다.

따라서 필요한 연관 데이터와 조회량을 보고 선택해야 합니다. 특히 여러 컬렉션의 동시 페치와 페이징은 실제 SQL과 결과 행 수를 확인해야 합니다. [Hibernate 7.4의 페치 조인 설명](#), [limit과 페치 조인에 관한 주의](#)

컴파일러의 인라이닝과도 비슷합니다. 함수를 펼쳐 넣는 것이 도움이 될 수 있지만, 지나치면 코드 크기가 커지는 비용이 생기죠. 최적화에는 효과와 비용이 함께 있습니다.

**사전 확인:** MySQL과 PostgreSQL 방언은 7.4에서 둘 다 서브쿼리 안의 offset을 지원한다고 선언한다( `supportsOffsetInSubquery` ). 컬렉션 페치 조인에 `setMaxResults` 를 건 쿼리가 실제로 어떤 SQL로 바뀌는지는 로그로 확인한 뒤 말한다. 확인했다면 "같은 JPQL도 하이버네이트 버전에 따라 다른 구조로 lowering된다"는 사례로 연결할 수 있다.

## C-3. "쿼리 플랜 캐시가 있으면 파싱 비용은 신경 쓰지 않아도 되나요?"

같은 쿼리의 해석 결과를 재사용할 수 있다면 반복적인 파싱 비용을 줄일 수 있습니다. 다만 캐시를 설명할 때는 어느 층의 캐시인지 구분해야 합니다.

| 구분                      | 주로 재사용하는 대상         | 위치        |
|-------------------------|---------------------|-----------|
| 하이버네이트 쿼리 해석과 플랜 캐시     | 쿼리 해석 결과와 실행 준비 정보  | JVM       |
| JDBC 드라이버의 statement 캐시 | 준비된 statement 관련 자원 | 드라이버와 커넥션 |
| DB 실행 계획 캐시             | SQL 실행 계획           | DB 서버     |

하이버네이트에서는 쿼리 문자열이 해석 결과를 재사용하는 데 중요한 기준이 됩니다. 값을 매번 문자열에 직접 넣으면 서로 다른 쿼리가 늘어나고, 캐시 활용도가 떨어질 수 있습니다. 값을 파라미터로 전달하면 쿼리 형태를 유지하기가 쉽습니다. 하이버네이트 7.4에서 Criteria 쿼리는 기본 설정으로는 이 캐시를 쓰지 않는다는 점도 함께 기억해둡니다(B-5).

DB 실행 계획의 재사용에는 또 다른 고려 사항이 있습니다. SQL Server는 컴파일할 때 파라미터 값을 참고해 계획을 만들 수 있습니다. 이를 parameter sniffing이라고 합니다. [SQL Server의 쿼리 처리 설명](#)

이때 특정 값에 적합했던 계획을 다른 값에도 재사용하면 성능이 나빠질 수 있습니다. 계획을 다시 만드는 비용과, 현재 값에 더 잘 맞는 계획을 사용하는 이점 사이에서 균형이 필요한 겁니다. [Microsoft의 파라미터 민감도 설명](#)

**답변 요령:** 세 캐시를 먼저 구분한 뒤 설명한다. 파싱 결과의 재사용, statement의 재사용, DB 실행 계획의 재사용을 같은 문제로 섞지 않는다.

## D. 발표 범위와 비교

### D-1. "영속성 컨텍스트까지 컴파일러라고 볼 수는 없지 않나요?"

맞습니다. 오늘 다룬 범위는 하이버네이트 안에서 쿼리를 해석하고 SQL로 변환하는 부분입니다.

하이버네이트는 이 밖에도 객체를 관리하고, 변경을 감지하고, DB에 반영할 작업을 정리합니다. 이를 언어 구현체에 빗대면 컴파일러와 런타임의 역할을 함께 가진 것으로 볼 수 있습니다.

| 하이버네이트의 기능       | 역할을 이해하기 위한 비유          |
|------------------|-------------------------|
| JPQL의 해석과 SQL 변환 | 컴파일러                    |
| 영속성 컨텍스트         | 실행 중 객체를 관리하는 영역        |
| 더티 체크            | 객체의 변경을 감지하는 장치         |
| flush 처리         | 모아둔 변경을 DB 작업으로 반영하는 단계 |
| 쿼리 플랜 캐시         | 준비된 실행 정보를 재사용하는 캐시     |

이 표는 역할을 이해하기 위한 비유입니다. 각 기능이 컴파일러나 런타임의 특정 기능과 정확히 일치한다는 의미는 아닙니다.

**답변 요령:** 발표에서 다룬 범위를 다시 짚는다. 쿼리 변환과 객체 관리가 함께 존재한다는 점을 설명하면 충분하다.

## D-2. "Hibernate 5와는 무엇이 다른가요?"

하이버네이트 6에서 쿼리 처리 구조가 크게 바뀌었습니다. ANTLR 4 기반의 파싱과 함께, SQM과 SQL AST를 중심으로 쿼리를 처리하는 구조가 들어왔습니다. 이 발표의 기준인 7.4도 이 구조를 이어받고 있습니다.

차이가 가장 잘 드러나는 곳은 Criteria입니다. 하이버네이트 5에서는 Criteria 쿼리를 JPQL 문자열로 렌더링한 뒤, 그 문자열을 다시 파싱했습니다. 6부터는 Criteria로 조립한 트리가 곧 SQM이라서 이 왕복이 사라졌습니다. 공통된 중간 표현이 있으면 입력 경로끼리 문자열로 돌아갈 필요가 없어진다는 사례입니다.

그런데 Spring Data JPA 4는 메서드 이름 쿼리를 다시 JPQL 문자열로 만듭니다. 왕복을 없앨 수 있는데도 문자열을 택한 이유는 B-5에서 설명한 재사용의 이득입니다.

발표의 관점에서는 파서가 바뀌었다는 사실보다, **쿼리의 의미를 표현하는 단계와 SQL의 구조를 만드는 단계를 구분할 수 있다**는 점에 주목했습니다. 이전 버전의 구체적인 구현과 비교할 때는 해당 버전의 소스를 함께 확인해야 합니다.

**사전 확인:** 하이버네이트 5의 Criteria 경로(`CriteriaQueryImpl` 이 JPQL 문자열을 렌더링 하고, 그 문자열로 쿼리를 다시 만드는 부분)를 5.6 소스에서 한 번 확인해둔다.

## D-3. "MyBatis와는 어떻게 다른가요?"

MyBatis에서는 개발자가 SQL을 직접 작성합니다. 따라서 하이버네이트처럼 엔티티를 대상으로 한 쿼리를 해석해서 SQL로 바꾸는 과정이 중심이 되지 않습니다.

동적 SQL을 구성하거나 파라미터와 결과를 매핑하는 기능은 있지만, 오늘 설명한 SQM 기반의 변환과는 역할이 다릅니다.

핵심 차이는 어느 부분을 개발자가 직접 표현하느냐입니다. 하이버네이트에서는 객체 모델과 매핑을 바탕으로 SQL을 생성하고, MyBatis에서는 개발자가 작성한 SQL을 중심으로 동작합니다.

## E. 상속 매핑과 lowering

5파트의 상속 예제에서 이어지는 질문에 대비한 답변이다. 실습 결과가 들어갈 자리는 비워두었다.

### E-1. "상속 말고, SQL에 없는 개념을 흉내 내는 다른 예도 있나요?"

`TYPE()` 연산자가 좋은 예입니다. `WHERE TYPE(c) = Waiting` 처럼 쓰면, 조회 대상이 어떤 자바 타입인지로 조건을 걸 수 있습니다.

SQL에는 "이 행의 자바 타입"이라는 개념이 없습니다. 그래서 하이버네이트는 이 조건을 상속 전략에 맞는 SQL 조각으로 바꿉니다.

[실습 6 결과 삽입: 전략별로 `TYPE()` 조건이 바뀐 SQL 조각]

## E-2. "UNION ALL 쿼리에 있는 `clazz_` 는 무엇인가요?"

`TABLE_PER_CLASS` 전략에서는 여러 테이블을 UNION ALL로 합칩니다. 합친 결과의 행만 보서는 그 행이 어느 테이블에서 왔는지 알 수 없죠.

그래서 하이버네이트는 서브쿼리마다 상수 값을 하나씩 넣어, `clazz_` 라는 이름으로 타입을 표시합니다.

`SINGLE_TABLE`의 판별 컬럼은 테이블에 저장된 값입니다. 반면 `clazz_`는 쿼리를 실행하는 시점에 만들어지는 값이라는 차이가 있습니다.

[비유 삽입: 구현 문서 B13에서 정한 비유]

## 실습 결과와 슬라이드 연결

상세한 실습 절차는 별도 문서인 "하이버네이트 테코톡 실습 문서"를 참고한다. 아래 표는 각 결과를 발표의 어느 부분에 사용할지 정리한 것이다.

| 실습 | 결과물                                                             | 사용할 위치   |
|----|-----------------------------------------------------------------|----------|
| 1  | 세 입력 경로의 SQM 덤프 비교와 경로 그림(메서드 이름이 JPQL 문자열을 거치는 경로)             | 2-2, B-3 |
| 2  | MySQL과 PostgreSQL의 SQL 비교, <code>SqlAstTranslator</code> 구현체 목록 | 1파트, 4-3 |
| 3  | 오타가 있는 JPQL의 예외와 SQL이 실행되지 않은 로그                                | 3-2      |
| 4  | 세 상속 전략의 SQL과 SQM 덤프 비교                                         | 5파트      |
| 5  | SQL AST와 <code>DomainResult</code> 덤프                           | 6파트      |

실습 7의 N+1 조건별 측정 결과와 실습 8의 쿼리 플랜 캐시 결과는 질의응답용 보조 슬라이드에 둔다. 실습 7은 C-1, 실습 8은 B-5와 C-3 질문이 나왔을 때 활용한다. 실습 6의 `TYPE()` 결과는 E-1에서 쓴다.

실습 4에서는 **SQM 덤프 비교를 반드시 수행한다**. 상속 전략이 달라져도 어떤 조회 구조가 유지되는지 확인해야 하기 때문이다. 확인한 범위에 맞춰 슬라이드와 설명을 작성한다.

## 발표 전에 소스에서 확인할 항목

주요 클래스와 처리 경로를 직접 확인해두면 설명을 구체적으로 뒷받침할 수 있다. 패키지 와 메서드는 발표에서 사용하는 버전(Hibernate ORM 7.4, Spring Data JPA 4.1)을 기준으로 확인한다.

| 단계                             | 클래스 또는 파일                                                                                                                                                    |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 메서드 이름에서 JPQL 생성 (Spring Data) | <code>PartTreeJpaQuery</code> , <code>JpaQueryCreator</code> , <code>JpqlQueryBuilder</code>                                                                 |
| HQL 문법 정의                      | <code>HqlParser.g4</code>                                                                                                                                    |
| 파싱 결과에서 SQM 생성                 | <code>SemanticQueryBuilder</code>                                                                                                                            |
| Criteria API로 SQM 조립           | <code>SqmCriteriaNodeBuilder</code> , <code>SqmFrom</code> 과 <code>JPA From</code> 의 인터페이스 관계                                                                |
| SQM 트리                         | <code>org.hibernate.query.sqm</code> 패키지                                                                                                                     |
| SQM에서 SQL AST로 변환              | <code>StandardSqmTranslator</code> 와 관련 변환 클래스                                                                                                               |
| SQL AST 트리                     | <code>org.hibernate.sql.ast</code> 패키지                                                                                                                       |
| SQL AST에서 DB별 SQL 생성           | <code>SqlAstTranslator</code> 인터페이스와 구현체                                                                                                                     |
| 변환기 팩터리 선택                     | <code>Dialect#getSqmTranslatorFactory</code> , <code>Dialect#getSqlAstTranslatorFactory</code> 와 실제 호출 경로                                                    |
| 쿼리 플랜 캐시 (B-5, C-3)            | <code>QuerySettings</code> 의 <code>QUERY_PLAN_CACHE_ENABLED</code> 와 <code>CRITERIA_PLAN_CACHE_ENABLED</code> , 캐시 키를 만드는 <code>SqmInterpretationsKey</code> |

IDE에서 `SqlAstTranslator` 의 구현체 목록을 펼친 화면을 준비하면, 4-3에서 DB별 SQL 생성기를 설명할 때 활용하기 좋다.

## 리허설 체크리스트

- ☐ 도입에서 쿼리의 처리 구조를 살펴보는 발표라는 점을 전달했는가?
- ☐ 도입에서는 N+1을 언급하지 않고 핵심 질문에 집중했는가?

- ☐ MySQL과 PostgreSQL의 페이징 SQL을 `setMaxResults(10)` 조건의 실제 로그로 교체했는가?
- ☐ 2-2의 그림에 메서드 이름이 JPQL 문자열을 거쳐 HQL 경로로 합류하는 흐름이 드러나는가?
- ☐ 3-3에서 SQM을 고수준 중간 표현의 역할에 빗대어 설명했는가?
- ☐ 4-3에서 도입의 질문에 분명하게 답했는가?
- ☐ 5파트에서는 세 SQL의 구조 차이만 짚었는가?
- ☐ 5파트를 1분 30초 안에 마쳤는가?
- ☐ 상속 전략별 SQM 덤프를 비교하고, 확인한 범위에 맞게 설명했는가?
- ☐ 7파트를 40초 안에 마쳤는가?
- ☐ N+1의 자세한 설명은 질의응답으로 이어가겠다고 안내했는가?
- ☐ N+1 측정 표는 질의응답용 보조 슬라이드에 두었는가?
- ☐ 마무리의 핵심 문장을 익혔는가?
- ☐ ALL(\*)의 세부 설명은 질의응답 자료에만 두었는가?
- ☐ 실제 낭독 시간을 측정했는가? 10분 발표에서 30초의 여유를 두려면 9분 30초 안에 마치는 것을 목표로 한다.

## 부록 A. 발표의 중심을 옮긴 이유

발표자가 전체 방향을 확인하기 위한 메모다.

이전 버전은 제목에서 JPQL이 SQL로 변환되는 과정을 예고했지만, 본문의 중심은 연관 엔티티를 언제, 어떻게 가져오는지에 있었다. 전자는 쿼리의 해석과 변환에 관한 질문이고, 후자는 페치 계획과 실행 시점에 관한 질문이다.

SQM까지 설명한 뒤 곧바로 영속성 컨텍스트와 N+1에 많은 시간을 쓰면, 청중의 관심도 쿼리 변환에서 성능 문제로 옮겨갈 수 있다. 10분 안에 핵심을 전달하려면 SQM과 lowering을 중심으로 설명하고, N+1은 응용 사례로 짧게 연결하는 편이 적절하다.

도입도 같은 이유로 바꿨다. EAGER 설정에서 N+1이 발생하는 사례는 조회 경로와 매핑 조건을 함께 설명해야 한다. DB별 페이징 SQL을 비교하면, 같은 조회 요청이 서로 다른 SQL로 표현된다는 점을 곧바로 보여줄 수 있다. 다만 이 사례도 발표 환경에서 실제 SQL을 확인한 뒤 사용한다.

## 부록 B. 이전 구성에서 달라진 점

| 항목     | 이전 구성                | 현재 구성                                                | 변경 이유                                |
|--------|----------------------|------------------------------------------------------|--------------------------------------|
| 중심 내용  | N+1에 3분 배정           | SQM과 lowering에 4분 배정                                 | 제목에서 제시한 질문에 집중한다.                   |
| 도입     | EAGER 설정에서도 발생하는 N+1 | DB별 페이징 SQL 비교                                       | 같은 조회가 다른 SQL로 표현되는 모습을 보여준다.        |
| SQM    | 40초 정도의 연결 설명        | 2분의 독립된 파트                                           | 개념과 의미 분석의 근거를 함께 설명한다.              |
| 상속 예제  | 여러 예제 중 하나로 1분 배정    | 핵심 사례로 1분 30초 배정                                     | 매핑에 따라 SQL 구조가 달라지는 과정을 보여준다.        |
| N+1    | 발표의 핵심 사례            | 40초의 응용 사례                                           | 쿼리 처리 구조를 이해한 뒤 연결할 수 있는 주제로 둔다.     |
| IR 설명  | IR의 정의로 단정           | 고수준 중간 표현의 역할에 비유                                    | 용어의 범위보다 실제 역할에 초점을 맞춘다.             |
| 입력 경로  | 세 입력이 나란히 SQM으로 연결   | 메서드 이름이 JPQL 문자열을 거쳐 HQL 경로로 합류하는 흐름을 표시             | Spring Data JPA 4.x의 실제 처리 경로를 드러낸다. |
| 기준 버전  | Hibernate 6.6 문서 기준  | Spring Boot 4.1 (Hibernate 7.4, Spring Data JPA 4.1) | 실습 문서, 구현 문서와 같은 환경에서 확인한 내용으로 말한다.  |
| SQL 생성 | Dialect를 짧게 언급       | SqlAstTranslator 구현체 목록을 제시                          | DB별 SQL 생성 구조를 코드와 연결한다.             |
| 실습     | 4개                   | 오타 실습을 포함한 5개                                        | DB에 보내기 전에 의미를 분석한다는 점을 보여준다.        |
| 제목     | 부제 없음                | "SQM에서 SQL AST까지"라는 부제 추가                            | 발표가 다루는 범위를 미리 전달한다.                 |

## 편집 메모

원문의 8개 발표 파트와 코드 예제, 예상 질문의 주제를 유지하면서 문장 길이와 연결 표현을 다듬었다. 발표 본문과 답변은 말하기 편한 존댓말로, 준비 메모는 간결한 서술형으로 정리했다.

특히 "노드에 박혀 있다"는 "의미 분석 결과가 담겨 있다"로, "타겟의 재료로 흉내 낸다"는 "DB가 처리할 수 있는 구조로 구현한다"로 풀어썼다. lowering을 소개하는 핵심 문장도 같은 방향으로 통일했다.

표현만 고치면 오해가 남을 수 있는 부분은 설명 범위를 조정했다. "문자열이 한 번도 등장하지 않는다"는 "중간에 JPQL 문자열을 만들지 않는다"로, "SQM에서 노드 하나다"는 "SQM에서 같은 엔티티를 조회 대상으로 표현한다"로 구체화했다. 컴파일 시점도 자바 컴파일 시점과 구분했다.

암묵적 조인, EAGER 조회, 여러 컬렉션의 페치 조인, parameter sniffing은 관련 공식 문서를 확인해 설명을 바로잡고 해당 위치에 출처를 덧붙였다. 프로젝트별 SQL 로그와 SQM 덤프를 확인해야 하는 항목은 발표 준비 메모로 남겼다.

## v4 변경 사항

---

기준 버전을 Spring Boot 4.1(Hibernate ORM 7.4, Spring Data JPA 4.1)로 올려 실습 문서, 구현 문서와 맞췄다. 달라진 점은 다음과 같다.

- **2-2와 B-3:** Spring Data JPA 4.x는 메서드 이름 쿼리를 Criteria가 아니라 JPQL 문자열로 만든다. 그래서 v3의 "중간에 JPQL 문자열을 만들지 않고도 SQM에 도달합니다"는 삭제하고, "메서드 이름 쿼리는 두 번 파싱된다"로 바꿨다. 슬라이드의 합류 지점도 Criteria에서 HQL 입구로 옮겼다.
- **1파트:** PostgreSQL 쪽 SQL을 `fetch first ? rows only` 로 고쳤다. v3는 MySQL에는 `offset`이 없고 PostgreSQL에만 `offset`이 있어 두 SQL의 조건이 맞지 않았다. 예제 조건은 실습 2와 같은 `setMaxResults(10)` 으로 고정했다.
- **C-2:** 하이버네이트 7.4부터 컬렉션 페치 조인과 페이징을 함께 써도, 서브쿼리 안의 `limit`과 `offset`을 지원하는 DB에서는 메모리 페이징을 하지 않는다. 설명을 이에 맞췄다.
- **B-5, B-6, E-1, E-2 추가:** 실습 문서가 준비하라고 한 질의응답 중 대본에 없던 것을 뼈대로 추가했다. 실습 결과가 들어갈 자리는 비워두었다.
- **D-2:** 하이버네이트 5의 Criteria 문자열 왕복이 6에서 사라졌다는 사례를 넣었다.
- **문서 링크:** 6.6 문서를 7.4 문서로 교체했다. 7.4 가이드에서는 페치 조인 절의 앵커가 `explicit-fetch-join` 이다.
- **시간 배분:** 표의 합계 10:00과 리허설 목표 9:30이 어긋나 있어, 표를 상한으로 보고 9:30을 목표로 한다는 문장을 넣었다.